

TMG-Q

Calibration-Aware Mixed-Precision Quantization with Physical 2/3/4-bit Weight Packing

Abdullah Salem Saleh Al-Faqeer

Independent Researcher | June 2026

github.com/abdallah2183/TMG-Q-Tanh-Mixed-Genetic-Quantization-Framework

TMG-Q end-to-end compression pipeline

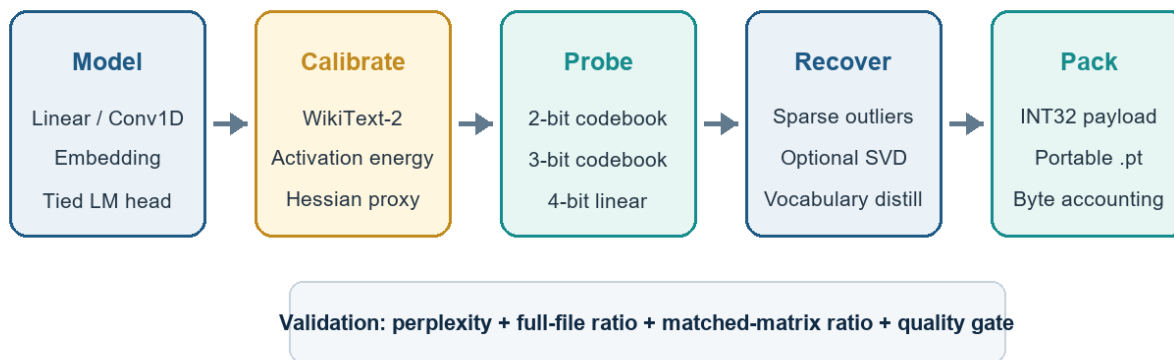


Figure 1. TMG-Q pipeline from model calibration to physical packing and measured validation.

3.51x GPT-2 full-file compression	+4.61% PPL change at that point	2.70x TinyLlama full-file compression	32 x 128 WikiText-2 evaluation chunks
---	---	---	---

Abstract

We present TMG-Q, an experimental post-training quantization framework for causal language models that combines physical INT32 bit packing, calibration-derived sensitivity estimates, learned group codebooks, sparse residual recovery, and adaptive mixed precision. The implementation supports 2-, 3-, and 4-bit weight storage for linear, GPT-2 Conv1D, embedding, and tied language-model-head modules. Unlike theoretical bit-count reports, TMG-Q measures both the serialized checkpoint and the payload of matched quantized matrices. Reproduced experiments on an NVIDIA RTX 5060 Ti show two practical GPT-2 Base operating points: 3.19x full-file compression with a 0.39% perplexity increase, and 3.51x compression with a 4.61% increase. On TinyLlama 1.1B, adaptive 2/3/4-bit allocation reaches 2.70x full-file compression and 3.51x compression over matched matrices, with perplexity increasing from 17.7996 to 18.9206. The results establish a reproducible research baseline while showing that robust 2-bit operation and optimized packed inference remain open problems.

Keywords: large language models, post-training quantization, mixed precision, codebook quantization, model compression, perplexity.

1. Introduction

Autoregressive language models are expensive to store and move through memory because every generated token requires repeated access to large parameter tensors. Weight-only post-training quantization (PTQ) addresses this bottleneck by replacing floating-point weights with low-bit representations while retaining higher-precision activations. The practical objective is not simply to minimize reconstruction error: a usable method must jointly manage model quality, checkpoint bytes, calibration cost, architectural compatibility, and runtime support.

TMG-Q was developed as an end-to-end laboratory for this trade-off. Its modern path quantizes Hugging Face causal language models on CUDA, emits portable .pt checkpoints, reloads the packed modules for evaluation, and reports actual serialized size. This paper focuses only on results reproduced with that path. Earlier project observations for GPT-2 Medium, GPT-2 Large, NanoGPT, and LLaVA are excluded from the primary evidence because they were produced by different scripts or evaluation protocols.

1.1 Contributions

Physical low-bit storage. A common failure in experimental quantization is to retain integer values in byte-addressable tensors while claiming 2- or 3-bit storage. TMG-Q packs indices into INT32 containers: 16 values at 2-bit, 10 at 3-bit, and 8 at 4-bit.

Adaptive precision selection. Each projection is probed under 2-bit codebook, 3-bit codebook, and 4-bit linear candidates. The lowest precision satisfying a Hessian-weighted normalized error threshold is selected.

Recovery mechanisms. The framework supports row-wise clipping, sparse high-precision outlier residuals, optional low-rank SVD residuals, and low-rank vocabulary logit distillation.

Reproducible accounting. Evaluation reports perplexity, complete checkpoint ratio, matched-matrix ratio, bit allocation, and explicit rejection when a configured quality gate is exceeded.

Scope. The present evidence supports checkpoint compression, not inference acceleration. Packed weights are dequantized by PyTorch modules; a fused low-bit CUDA matrix-multiplication kernel has not yet been implemented.

2. Related Work

GPTQ introduced a scalable one-shot method based on approximate second-order information and demonstrated accurate 3- and 4-bit weight quantization [1]. AWQ observed that activation magnitude reveals salient channels and protects them through equivalent scaling, pairing the method with optimized 4-bit inference kernels [2]. TMG-Q shares the use of calibration statistics but differs in selecting among heterogeneous 2/3/4-bit candidate encodings and in exposing full serialized accounting.

Extreme compression methods increasingly use richer representations. AQLM applies learned additive codebooks and block-level optimization below 3 bits per parameter [3]. QuIP# combines randomized Hadamard incoherence processing with lattice codebooks and fine-tuning [4]. These works indicate why uniform scalar 2-bit rounding is insufficient, a result also observed directly in TMG-Q.

Sparse recovery is another established direction. SpQR isolates high-impact outliers in higher precision [5], while SqueezeLLM combines sensitivity-based non-uniform quantization with a dense-and-sparse decomposition [6]. TMG-Q's sparse residual mechanism follows the same broad principle, but its current implementation remains a research prototype rather than an optimized inference format.

Method	Primary mechanism	Typical regime
GPTQ	Approx. second-order updates	3-4 bit
AWQ	Activation-aware channel scaling	4 bit
AQLM	Additive learned codebooks	2-3 bit
QuIP#	Incoherence + lattice codebooks	2-4 bit
SpQR / SqueezeLLM	Sparse outlier isolation	3-4 bit
TMG-Q	Adaptive codebook/linear + residuals	2/3/4 bit

Table 1. Positioning of TMG-Q relative to representative weight-only PTQ methods.

2.1 Research Gap and Design Goals

Published systems demonstrate strong quality, but reproducing their full stack can require specialized kernels, block reconstruction, or vector codebooks. TMG-Q asks a narrower engineering question: how far can a transparent PyTorch research pipeline push real checkpoint compression while preserving auditable byte accounting and a direct path from export to perplexity evaluation?

The design therefore prioritizes four properties: (i) every claimed low-bit index is physically packed; (ii) layer choices are derived from calibration rather than fixed position rules; (iii) failed configurations remain visible in the experiment record; and (iv) a compressed checkpoint can be reloaded without the original floating-point weights.

Research questions. RQ1: Can adaptive 2/3/4-bit allocation improve the compression-quality frontier over fixed policies? RQ2: Which recovery mechanisms are necessary below 4-bit? RQ3: How much does complete checkpoint accounting differ from matrix-only accounting?

3. Method

3.1 Grouped Quantization and Packing

For a weight row divided into groups of size g , the linear path estimates a scale and offset after clipping search. A weight w is mapped to an integer index q in the range $[0, 2^b-1]$. The codebook path instead learns 2^b centroids for each group by Lloyd-style updates. At load time, indices select centroids or reconstruct affine values.

Linear reconstruction: $q = \text{clip}(\text{round}(w / s) + z)$, and $w\text{-hat} = (q - z)s$.
Codebook reconstruction: $w\text{-hat} = C[q]$, where C stores group-specific learned centroids.

The integer indices are packed with bitwise shifts into INT32 tensors. Metadata, codebooks, residuals, shape descriptors, and alignment waste are included in the measured payload. Three-bit storage uses 30 of every 32 container bits, so its practical size is slightly above an idealized three-bit stream.

Bits	Values / INT32	Index utilization	Candidate
2	16	32 / 32	Learned codebook
3	10	30 / 32	Learned codebook
4	8	32 / 32	Affine linear

Table 2. Physical index packing used in the modern TMG-Q checkpoint format.

3.2 Calibration and Sensitivity

Calibration forwards WikiText-2 token blocks through the uncompressed model and accumulates input-channel energy for each projection. These statistics act as a diagonal Hessian proxy. Candidate reconstruction error is weighted by this proxy so that errors on frequently excited channels receive greater penalty.

Selection metric: $\text{NMSE}_H = \text{sum}(H * (W - W\text{-hat})^2) / \text{max}(\text{sum}(H * W^2), \text{epsilon})$.

3.3 Adaptive and Budget Policies

The adaptive policy tests 2-bit codebook, 3-bit codebook, and 4-bit linear candidates on representative rows. It accepts the first candidate below its threshold; otherwise it chooses the lowest-error candidate. A minimum-bit constraint can disable unstable regimes. The budget policy estimates candidate payloads for all layers and optimizes a global assignment under a target ratio, rejecting mathematically infeasible requests before full export.

3.4 Residual and Vocabulary Recovery

A sparse residual stores a configurable fraction of the largest reconstruction errors as row, column, and value triples. Optional SVD factors approximate structured residual error, although the tested TinyLlama configurations did not justify their additional bytes. For GPT-2, quantizing the tied vocabulary directly caused severe degradation. TMG-Q therefore freezes all packed transformer weights and trains a low-rank correction against teacher logits. This localized distillation recovered vocabulary quality without restoring the full embedding matrix.

Calibration and residual recovery prevent sub-4-bit collapse

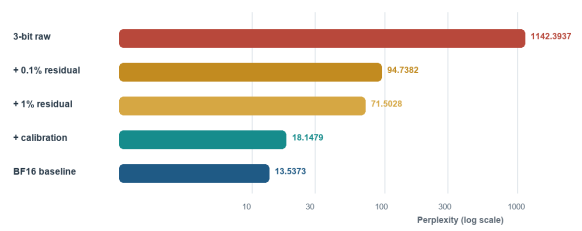


Figure 2. Quick-sweep TinyLlama evidence that calibration and sparse recovery are essential at 3-bit.

3.5 Export Algorithm

Step	Operation
1	Load BF16 model and tokenizer; enumerate quantizable modules.
2	Run calibration blocks and accumulate per-channel energy.
3	Probe low-bit candidates on representative rows.
4	Select precision adaptively or solve the global payload budget.
5	Quantize the full tensor; attach sparse/SVD recovery if enabled.
6	Pack indices, save metadata, reload, evaluate, and measure bytes.

Algorithm 1. High-level TMG-Q export and verification sequence.

3.6 Complexity and Implementation

Quantization proceeds one module at a time, limiting peak device memory to the model plus temporary candidate tensors. Adaptive probing reduces candidate cost by evaluating sampled rows before one complete quantization pass. Codebook updates scale with the number of groups, centroids, iterations, and probed elements; linear clipping search is cheaper. The exported state dictionary stores only packed modules, required metadata, and any selected residual parameters.

The loader reconstructs quantized module classes dynamically from shape and bit-width metadata. This architectural replacement is important for GPT-2 because its projections use the transposed Conv1D convention rather than standard PyTorch Linear storage.

4. Experimental Protocol

Experiments were executed locally on an NVIDIA GeForce RTX 5060 Ti with 16 GB VRAM using a CUDA-enabled PyTorch 2.12 development build and Hugging Face Transformers. GPT-2 Base and TinyLlama-1.1B-Chat-v1.0 were evaluated on the WikiText-2 test split [7]. The reported verification protocol uses 32 non-overlapping chunks of 128 tokens in BF16 runtime. Calibration uses separate training-split chunks.

Perplexity is computed by teacher-forced causal language-model loss. Complete checkpoint compression compares the serialized TMG-Q file with the model's unique FP16 parameter payload. Matched-matrix compression compares only tensors replaced by quantized modules. This distinction prevents embeddings, preserved parameters, and metadata from being hidden behind a theoretical bit count.

Item	GPT-2 Base	TinyLlama 1.1B
Evaluation	32 x 128 tokens	32 x 128 tokens
Runtime dtype	BF16	BF16
Calibration	WikiText-2 train	WikiText-2 train
Quantized modules	49	154
Hardware	RTX 5060 Ti 16 GB	RTX 5060 Ti 16 GB

Table 3. Reproduced evaluation environment and scope.

5. Results

5.1 GPT-2 Base

GPT-2 Base: measured compression-quality operating points

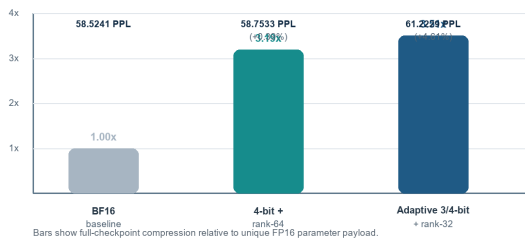


Figure 3. Full-file compression and perplexity for the verified GPT-2 operating points.

The quality-oriented checkpoint uses 4-bit packed weights and a rank-64 distilled vocabulary. It reduces the complete file from approximately 237.4 MiB to 74.5 MiB (3.19x) while increasing perplexity from 58.5241 to 58.7533, a relative change of 0.39%. The compression-oriented checkpoint uses two 3-bit and 47 4-bit matrices plus a rank-32 vocabulary correction. Its 67.7 MiB file reaches 3.51x compression with perplexity 61.2229, a 4.61% increase.

Operating point	File	Ratio	PPL	Relative change
BF16 baseline	237.4 MiB	1.00x	58.5241	-
4-bit + rank-64	74.5 MiB	3.19x	58.7533	+0.39%
Adaptive 3/4-bit + rank-32	67.7 MiB	3.51x	61.2229	+4.61%

Table 4. GPT-2 Base results reproduced on June 7, 2026.

5.2 TinyLlama 1.1B

TinyLlama 1.1B: adaptive precision and real payload accounting

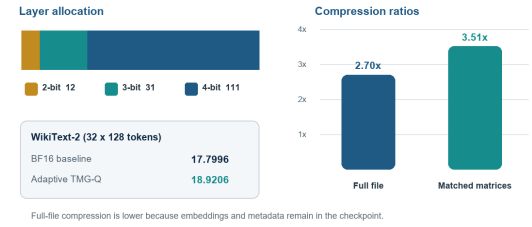


Figure 4. TinyLlama layer allocation, compression ratios, and verified perplexity.

Adaptive TinyLlama quantization assigns 12 projections to 2-bit codebooks, 31 to 3-bit codebooks, and 111 to 4-bit linear storage. With group size 64, calibration, and a 0.1% sparse residual, perplexity rises from 17.7996 to 18.9206 (+6.30%). The matched quantized matrices compress by 3.51x, while the complete checkpoint compresses by 2.70x because embeddings, preserved parameters, and metadata remain.

Metric	BF16	Adaptive TMG-Q
WikiText-2 PPL	17.7996	18.9206
Relative PPL change	-	+6.30%
Full payload	2098.2 MiB	778.0 MiB
Full-file ratio	1.00x	2.70x
Matched matrices	1848.0 MiB	526.9 MiB
Matched-matrix ratio	1.00x	3.51x

Table 5. TinyLlama full-file and matrix-level accounting.

Key observation. Reporting only the 3.51x matrix ratio would overstate deployable compression. The complete checkpoint ratio is 2.70x after preserved tensors and metadata are counted.

6. Analysis and Discussion

6.1 What Worked

The strongest result is not a single quantizer but a coordinated pipeline. Calibration consistently improved TinyLlama 4-bit quality and transformed the 3-bit linear path from collapse into a usable regime when combined with residual recovery. Learned codebooks were substantially better than linear quantization at 2 and 3 bits. Adaptive allocation then exploited easy layers without forcing the entire model into an unstable precision.

Vocabulary distillation was decisive for GPT-2. A fully tied 4-bit vocabulary without training produced perplexity above 1700 in the initial test, whereas a rank-64 correction returned the 32-chunk result to 58.7533. This indicates that embedding and output-head error can dominate checkpoint quality even when transformer projections are well reconstructed.

6.2 Negative Results

Uniform 2-bit linear quantization remains unusable for TinyLlama. A 1% residual and rank-8 SVD correction reduced damage but did not restore acceptable perplexity. Metadata compression was also fragile: experimental FP8 conversion of linear scales and offsets caused catastrophic error. QAT-lite reduced calibration divergence for selected codebooks but did not improve held-out WikiText-2 perplexity in the tested configurations.

Experiment	Result	Decision
3-bit raw, g128	PPL 1142.3937	Rejected
3-bit + 1% residual + calibration	PPL 18.1479	Recovered
2-bit + 1% residual + rank-8 SVD	PPL 431.8017	Rejected
GPT-2 rank-24 vocabulary	+7.91% PPL	Rejected
FP8 vocabulary residual	+5.02% PPL	Outside gate
FP8 linear scales / zeros	PPL 4622.9760	Rejected

Table 6. Representative negative results retained to make the research boundary explicit.

6.3 Limitations and Threats to Validity

The verification subset contains 4096 evaluated tokens per model and is therefore smaller than standardized full-corpus protocols. Results have not been independently reproduced. The two model families differ in architecture and tokenizer, so their perplexities cannot be compared directly. Checkpoint compression does not imply latency or energy improvement because no fused packed GEMM kernel is present. Finally, thresholds and calibration choices were tuned through local sweeps, creating a risk of selection bias.

6.4 Future Work

The next evaluation milestone is integration with lm-evaluation-harness and a standardized WikiText-2/C4 protocol on the same model families used by published baselines. Methodologically, randomized rotations or other incoherence processing should be evaluated before further claims at 2-bit. Additive or vector codebooks may also improve the accuracy-per-byte frontier beyond scalar group codebooks.

On the systems side, fused dequantization-GEMM CUDA kernels are required to test whether checkpoint savings translate into decoding speed. Safetensors or GGUF serialization, deterministic calibration manifests, and public checkpoint hashes would strengthen external reproducibility.

7. Conclusion

TMG-Q demonstrates that a transparent combination of physical packing, calibration-aware sensitivity, learned codebooks, sparse recovery, and localized distillation can produce meaningful low-bit compression on real language models. The best verified full-file result is 3.51x on GPT-2 Base within a 5% perplexity gate; TinyLlama reaches 2.70x full-file and 3.51x matched-matrix compression with a 6.30% perplexity increase. The next research priorities are standardized evaluation, rotation or incoherence processing for robust 2-bit quantization, and fused CUDA kernels that convert storage gains into measured inference speedups.

References

[1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," ICLR, 2023. arXiv:2210.17323.

[2] J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," MLSys, 2024. arXiv:2306.00978.

[3] V. Egiazarian et al., "Extreme Compression of Large Language Models via Additive Quantization," ICML, 2024. arXiv:2401.06118.

[4] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks," ICML, 2024. arXiv:2402.04396.

[5] T. Dettmers et al., "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," ICLR, 2024. arXiv:2306.03078.

[6] S. Kim et al., "SqueezeLLM: Dense-and-Sparse Quantization," ICML, 2024. arXiv:2306.07629.

[7] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," ICLR, 2017. arXiv:1609.07843.

Reproducibility artifact. Source code, evaluation utilities, experiment report, and checkpoint accounting are available in the public TMG-Q repository.